4

Setting Up Your Free Training Environment

Why This Chapter Exists

Everything in this book, from supervised fine-tuning in Chapter 5 to GRPO training in Chapter 12, requires running code on a GPU. Language models are too large to train on a laptop CPU. But you do not need to spend money. Free cloud platforms now provide GPU access that is powerful enough to fine-tune small language models, and that is exactly what we will set up in this chapter.

By the end of this chapter, you will have:

1. A working Google Colab notebook with GPU access
2. All required libraries installed (Unsloth, TRL, Hugging Face Transformers)
3. A language model loaded and running inference
4. Your first fine-tuning run completed (30 training steps, under 5 minutes)
5. Access to the companion code repository and its full notebook collection

Every code example in the rest of this book can be run in this environment.

The Companion Code Repository

All code for this book lives in one place:

```
https://github.com/PacktPublishing/Reinforcement-Learning-for-LLMs
```

Every chapter has a companion Jupyter notebook you can open directly in Google Colab. The repository mirrors the book's four-part structure:

```
Reinforcement-Learning-for-LLMs/
+-- notebooks/
|   +-- part1_foundations/    # Chapters 1-4
|   +-- part2_core/           # Chapters 5-10
|   +-- part3_advanced/       # Chapters 11-16
|   +-- part4_recipes/        # Chapters 17-19
+-- utils/
|   +-- data.py               # dataset loaders
|   +-- eval.py               # win rate, KL divergence
|   +-- viz.py                # training curve plots
+-- data/samples/             # toy datasets for offline runs
+-- requirements.txt
```

The `utils/` folder contains shared helpers imported automatically by the notebooks. You do not need to read this code directly.

Chapter-to-Notebook Map

Ch.	Title	Notebook
1	Essential Math Concepts	`part1_foundations/01_math_toolkit.ipynb`
2	Why LLMs Need RL	`part1_foundations/02_alignment_gap.ipynb`
3	RL Fundamentals	`part1_foundations/03_rl_fundamentals.ipynb`
4	Setting Up Your Environment	`part1_foundations/04_environment_setup.ipynb`
5	Supervised Fine-Tuning	`part2_core/05_sft.ipynb`

Ch.	Title	Notebook
6	RLHF and PPO	`part2_core/06_rlhf_ppo.ipynb`
7	Direct Preference Optimization	`part2_core/07_dpo.ipynb`
8	Online DPO	`part2_core/08_online_dpo.ipynb`
9	Reward Modeling	`part2_core/09_reward_modeling.ipynb`
10	Modern RL Algorithms	`part2_core/10_grpo_rloo_kto.ipynb`
11	Verifiers and Outcome Rewards	`part3_advanced/11_verifiers_outcome_rewards.ipynb`
12	Reasoning with GRPO (Concepts)	`part3_advanced/12a_reasoning_grpo_concepts.ipynb`
12	Reasoning with GRPO (DeepSeek)	`part3_advanced/12b_reasoning_grpo_deepseek.ipynb`
13	Test-Time Compute	`part3_advanced/13_test_time_compute.ipynb`
14	Self-Play and Constitutional AI	`part3_advanced/14_selfplay_constitutional_ai.ipynb`
15	Multi-Objective and Agentic RL	`part3_advanced/15_multiobjective_agentic.ipynb`
16	Domain-Specific RL	`part3_advanced/16_domain_specific_rl.ipynb`
17	Recipe: Aligned Chatbot	`part4_recipes/17_recipe_chatbot.ipynb`
18	Recipe: Reasoning Model	`part4_recipes/18_recipe_reasoner.ipynb`
19	Recipe: Agentic System	`part4_recipes/19_recipe_agent.ipynb`

Opening a notebook directly in Colab: On GitHub, navigate to any notebook, then replace `github.com` with `githubtocolab.com` in the URL. The notebook opens in Colab with no download required.

Cloning the Repository Locally

If you prefer to run notebooks locally with your own GPU or inside VS Code, clone the repository once:

```
git clone
https://github.com/PacktPublishing/Reinforcement-Learning-for-LLMs.git
cd Reinforcement-Learning-for-LLMs
pip install -r requirements.txt
```

Then open any notebook:

```
jupyter notebook notebooks/part2_core/05_sft.ipynb
```

Choosing a Platform

Two free platforms provide GPU access for training language models:

	Google Colab (Free)	**Kaggle Notebooks**
GPU	Tesla T4 (15 GB VRAM)	2× Tesla T4 (15 GB each)
Session limit	~12 hours	30 hours/week
Disk space	~100 GB	~20 GB
Ease of setup	Very easy	Easy
Best for	Quick experiments	Longer training runs

We will use **Google Colab** as our primary platform throughout this book. It is the simplest to get started with, and a single T4 GPU is more than enough for the models we will train. If you prefer Kaggle, the code is identical; only the initial setup differs slightly.

Setting Up Google Colab

Step 1: Open a New Notebook

1. Go to `colab.research.google.com`
2. Click **New Notebook**

3. Select **Runtime → Change runtime type**
4. Set **Hardware accelerator** to **T4 GPU**
5. Click **Save**

A note on the free tier: Sessions can be interrupted during peak demand. Save frequently. Colab Pro ($10/month) gives priority access but nothing in this book requires it.

Step 2: Verify GPU Access

In the first cell of your notebook, run:

```
!nvidia-smi
```

You should see output showing a Tesla T4 with approximately 15 GB of memory. If you see "No GPU" or a CPU-only message, go back to Runtime → Change runtime type and make sure T4 GPU is selected.

Installing the Libraries

We need three core libraries:

Library	What it does
Unsloth	Makes fine-tuning 2x faster with 70% less memory. Optimized model loading with LoRA (explained in Chapter 5).
TRL	Hugging Face trainers for SFT, DPO, and RL. Used throughout the book.
Datasets	Hugging Face library for loading and processing training data.

Run the following cell to install everything:

```
%%capture
import os
if "COLAB_" not in "".join(os.environ.keys()):
    !pip install unsloth
else:
    # Colab-optimized install (avoids conflicts)
    !pip install --no-deps \
        bitsandbytes accelerate xformers \
        peft trl triton \
        cut_cross_entropy unsloth_zoo
    !pip install sentencepiece protobuf \
        "datasets>=3.4.1" huggingface_hub
    !pip install --no-deps unsloth
```

The `%%capture` magic hides the lengthy install output. This cell takes 2 to 3 minutes. When it completes without error, you are ready to go.

Why the two-step install for Colab?

Unsloth patches several Hugging Face libraries internally to speed up training. The `--no-deps` flag prevents pip from reinstalling dependencies that Colab already provides (like PyTorch and CUDA), avoiding version conflicts and saving time. This is the install pattern recommended by the Unsloth team.

Loading Your First Model

We will use **Qwen2.5-1.5B-Instruct** as our base model throughout this book. Here is why:

- **Small enough:** 1.5 billion parameters fits easily on a free T4 with 4-bit quantization
- **Capable enough:** Can follow instructions and demonstrate the training dynamics we care about
- **Fully open:** No gated access, no license restrictions, no API key needed
- **Well supported:** Unsloth provides optimized versions for fast fine-tuning

Load the model:

```
from unsloth import FastLanguageModel
import torch

max_seq_length = 2048
model_name = "unsloth/Qwen2.5-1.5B-Instruct"

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=model_name,
    max_seq_length=max_seq_length,
    load_in_4bit=True,   # 4-bit to fit on free GPU
    load_in_8bit=False,
)

print("Model loaded successfully!")
print(f"Parameters: {model.num_parameters():,}")
```

This downloads the model (about 1 GB with 4-bit quantization) and loads it onto the GPU. The first run takes a minute or two; subsequent runs use the cached version.

What is 4-bit quantization?

The full model stores each parameter as a 16-bit number, requiring about 3 GB of GPU memory for 1.5 billion parameters. **Quantization** compresses each parameter to just 4 bits, cutting memory usage by roughly 4x. The model loses a tiny amount of precision, but at this scale the difference is negligible.

This is what makes training on a free GPU possible. Without quantization, even a 1.5B model would be tight on a T4. With it, we have plenty of room for the model, the optimizer, and the training data.

Running Inference: The Model Before Training

Before we train anything, let us see what the base model can already do. This gives us a "before" snapshot to compare against after fine-tuning.

```
from transformers import TextStreamer

# Switch to inference mode (faster generation)
FastLanguageModel.for_inference(model)

# Build a prompt using the chat template
messages = [
    {"role": "user",
     "content": "What is reinforcement learning?"}
]

inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
).to("cuda")

# Generate and stream the response
output = model.generate(
    input_ids=inputs,
    max_new_tokens=256,
    temperature=0.7,
    streamer=TextStreamer(tokenizer),
)
```

You should see the model stream its response token by token. The answer will be reasonable but generic: it has not been trained on our data yet.

Try a few more prompts to build intuition:

```
prompts = [
    "Explain gradient descent to a 10-year-old.",
    "Write a haiku about machine learning.",
    "What is the difference between a reward "
    "and a value function?",
]

for prompt in prompts:
    print(f"\n{'='*50}")
    print(f"PROMPT: {prompt}\n{'='*50}")
    msgs = [{"role": "user", "content": prompt}]
    ids = tokenizer.apply_chat_template(
        msgs,
        tokenize=True,
        add_generation_prompt=True,
        return_tensors="pt",
    ).to("cuda")
    model.generate(
        input_ids=ids,
        max_new_tokens=256,
        temperature=0.7,
        streamer=TextStreamer(tokenizer),
    )
```

Pay attention to the quality of the responses. Is the model helpful? Does it stay on topic? Does it make things up? These observations will become important when we start training in Chapter 5.

Your First Fine-Tuning Run

Let us go one step further and do a minimal fine-tuning run. The goal is not to build a production model. It is to verify that the entire training pipeline works end to end on your setup. Think of this as a “Hello, World!” for fine-tuning.

Step 1: Add LoRA Adapters

Instead of updating all 1.5 billion parameters (which would not fit in memory), we use **LoRA** (Low-Rank Adaptation). LoRA adds a small number of trainable parameters on top of the frozen base model. We will explain the math behind LoRA in Chapter 5. For now, just run this:

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=42,
    max_seq_length=max_seq_length,
)

# How many parameters are we actually training?
trainable = sum(
    p.numel() for p in model.parameters()
    if p.requires_grad
)
total = model.num_parameters()
pct = 100 * trainable / total
print(f"Training {trainable:,} / {total:,} "
      f"parameters ({pct:.1f}%)")
```

You will see that LoRA trains only about 1 to 2% of the total parameters. This is why fine-tuning fits on a free GPU.

Step 2: Prepare a Tiny Dataset

We will create a small dataset of instruction-response pairs directly in the notebook. In Chapter 5, we will use proper curated datasets. For now, five examples are enough to test the pipeline:

```
from datasets import Dataset

train_data = [
    {"messages": [
        {"role": "user",
         "content": "What is RL?"},
        {"role": "assistant",
         "content": "Reinforcement learning is a type "
         "of machine learning where an agent learns "
         "by interacting with an environment and "
         "receiving rewards for good actions."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "What is a policy in RL?"},
        {"role": "assistant",
         "content": "A policy is the strategy an agent "
         "uses to decide what action to take. It maps "
         "states to probability distributions "
         "over actions."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "What is a reward signal?"},
        {"role": "assistant",
         "content": "A reward signal is feedback from "
         "the environment telling the agent how good "
         "or bad its action was. The agent tries to "
         "maximize total reward."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "What is the value function?"},
```

```
        {"role": "assistant",
         "content": "The value function estimates the "
         "expected total future reward from a given "
         "state. It tells the agent how good its "
         "current situation is on average."}
    ]},
    {"messages": [
        {"role": "user",
         "content": "Difference between SFT and RLHF?"},
        {"role": "assistant",
         "content": "SFT teaches a model by showing it "
         "good examples to imitate. RLHF goes further "
         "by letting the model generate responses and "
         "learning from human preferences about which "
         "responses are better."}
    ]},
]

dataset = Dataset.from_list(train_data)
print(f"Dataset size: {len(dataset)} examples")
```

Step 3: Run the Trainer

```
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    args=SFTConfig(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        warmup_steps=5,
        max_steps=30,
        learning_rate=2e-4,
        logging_steps=1,
        output_dir="outputs",
```

```
        optim="adamw_8bit",
        seed=42,
        report_to="none",
        max_seq_length=max_seq_length,
    ),
)

print("Starting training...")
stats = trainer.train()
print(f"Done! Final loss: {stats.training_loss:.4f}")
```

You should see the loss printed at each step, decreasing over the 30 steps. The entire run takes about 1 to 2 minutes.

What just happened? You fine-tuned a 1.5 billion parameter language model on a free GPU in under 2 minutes. Five examples and 30 steps will not meaningfully change the model. But the pipeline works: model loaded, LoRA applied, data formatted, trainer ran, loss decreased. Every chapter from here on follows this same pattern with better data, more steps, and more sophisticated training algorithms.

Troubleshooting

Problem	Solution
"No GPU available"	Runtime → Change runtime type → T4 GPU. If none are free, try later or use Kaggle.
CUDA out of memory	Restart runtime, then rerun all cells. Confirm `load_in_4bit=True`.
Import errors after install	Restart runtime after installation. Colab needs a fresh restart for new packages.
Model download is slow	Normal for the first run. Subsequent runs use the cache.
Loss is not decreasing	With only 5 examples, this can happen. The real training in Chapter 5 uses proper datasets.

What We Have and What Comes Next

Here is what your environment now supports, and where each capability gets used:

Capability	Verified	Used in
GPU access and CUDA	✓	Every chapter
Unsloth model loading	✓	Every chapter
4-bit quantization	✓	Every chapter
LoRA adapter training	✓	Chapters 5 onward
TRL SFTTrainer	✓	Chapter 5
Chat template formatting	✓	Chapters 5, 6, 7
Model inference	✓	Every chapter

In the next chapter, we will use this exact environment to do our first real training run: supervised fine-tuning. We will load a curated dataset, teach the model to follow instructions properly, and introduce the cold start technique that primes the model for everything that follows.

The setup is done. The real work begins now.

The companion code for this chapter is at `github.com/arunpshankar/packt-final` – swap `github.com` for `githubtocolab.com` to open it directly in Colab. The diagram below provides a visual summary of the RLHF pipeline covered in this chapter.

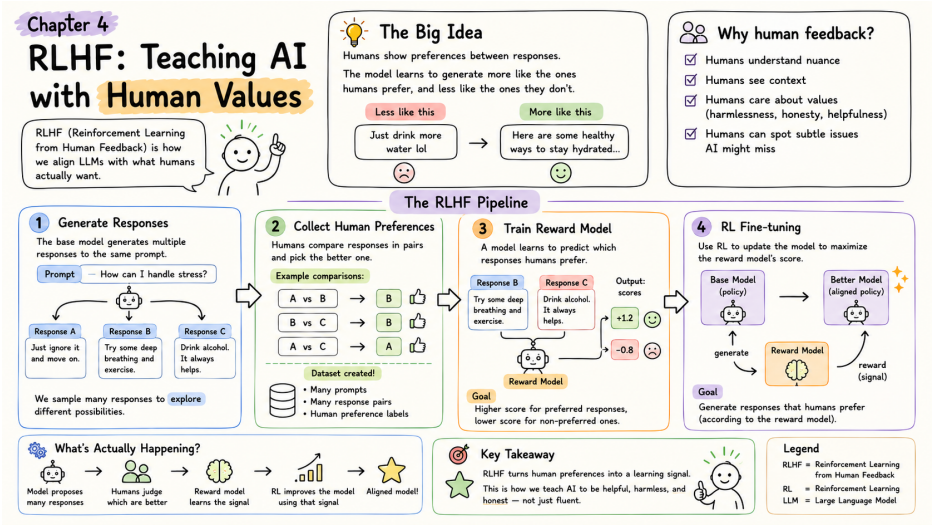

Figure 4.1: RLHF: teaching AI with human values – generate responses, collect preferences, train a reward model, and fine-tune with RL.